

RAPTOR Engine (engine.go) — Complete Explanation

This document provides a comprehensive explanation of the `engine.go` file, which implements the RAPTOR (Round-Based Public Transit Routing) algorithm. It is divided into three sections: a dictionary of core variables, a logical line-by-line breakdown, and a step-by-step dry run of a simple journey.

1. Core Variable Dictionary

Here is what each major variable in `RaptorSearch` holds and why it exists:

- **`buf (*state.SignalBuffer)`:** A read-only snapshot of the real-time seat availability for all trips. Captured once per search to ensure thread safety (no mid-search data mutations).
- **`routes (*schedule.RouteBuffer)`:** A read-only snapshot of the transit schedule (routes, stop times, footpaths). Also captured once.
- **`MaxRounds`:** The maximum number of trips a passenger can take. `MaxRounds = 4` means a passenger can take an initial train and make up to 3 transfers.
- **`tau ([]map[string]int64)`:** An array of maps. `tau[k][p]` holds the earliest known arrival time at station `p` using exactly `k` trips (rounds).
- **`bestArrival (map[string]int64)`:** This is $\tau^*(p)$ from the RAPTOR paper. It holds the absolute earliest arrival time at station `p` across *all* rounds. This is used to aggressively prune (skip) paths that are slower than a previously found path.
- **`marked / newMarked (map[string]bool)`:** RAPTOR only explores routes originating from stations that were *improved* in the previous round. `marked` tracks these stations.
- **`journal ([]map[string]journalEntry)`:** The breadcrumb trail. Whenever the engine finds a better arrival at a station, it records *how* it got there (which trip, where it boarded, etc.). Used at the end to backtrack from the destination and build the final itinerary.
- **`queue (map[int]int)`:** Maps a `routeIdx` to the *earliest stop position* on that route where a marked station exists. This ensures we don't scan a train line from the very beginning if we only boarded it halfway through.

2. Logical Line-by-Line Breakdown

Phase A: Initialization (Lines 73–113)

- **Lines 74-76:** The engine strictly enforces the "Snapshot Rule". It grabs the live pointers for seats (`buf`) and schedule (`routes`). Even if the background updaters change the schedule mid-search, this search relies only on the frozen snapshot.
- **Lines 88-100:** Allocates memory for `tau` and `journal`. There is one map per round (from 0 to `MaxRounds`).
- **Lines 103-108:** Sets the starting conditions. The `Origin` station is reached at `params.DepTime` in round 0. It is added to the `marked` list so the algorithm knows to start exploring from here.

Phase B: The Round Loop & Route Accumulation (Lines 114–132)

- **Line 114:** `for round := 1; round <= MaxRounds; round++` — RAPTOR operates in rounds. Round 1 finds all direct trains. Round 2 finds trains requiring 1 transfer, etc.
- **Lines 122-128:** It iterates over all `marked` stations (stations improved in the previous round). It finds every route passing through these stations. If a route passes through multiple marked stations, it only keeps the earliest boarding position (`minPos`) to avoid redundant scanning.

Phase C: Ride-Forward Scan (Lines 133–221)

This is the heart of the engine (Algorithm 1, Step 2 in the RAPTOR paper).

- **Line 138:** Iterate over every collected route.
- **Line 151-153:** `currentTrip` tracks the train we are currently riding. It starts as `nil` (we haven't boarded yet).
- **Line 155:** `for pos := boardPos; pos < len(route.StopIDs); pos++` — We travel forward along the route, stop by stop.
- **Lines 158-177 (Step A - Alighting):** If we are on a train (`currentTrip != nil`), we check if our arrival time at this stop is better than the previously known `bestArrival`. If it is, we update `tau`, update `bestArrival`, add the stop to `newMarked`, and record the journey in `journal`.
- **Lines 179-224 (Step B - Boarding/Switching):** Can we board a train here, or switch to a faster train?
 - `arrivalHere` is the earliest we could possibly arrive at this station.
 - The inner loop (`for ti := range routeTrips`) scans trips on this route.
 - We skip trips that depart *before* we arrive (`dep < arrivalHere`).
 - We skip trips that are slower than the train we are already on (`dep >= currentDep`).
 - We check seat availability (`canBoard`).
 - If a trip passes all checks, we board it: `currentTrip = tst`. The loop continues down the route on this new, better train.

Phase D: Footpath Relaxation (Lines 228–252)

- RAPTOR requires us to evaluate walking transfers *after* the transit scan.
- **Line 234-237:** We take a snapshot of `newMarked` (`transitImproved`).
- **Lines 238-252:** For every station improved by a train, we check its walkable neighbors (`routes.Footpaths`). If walking to the neighbor results in a faster arrival time, we update the neighbor's `bestArrival` and `tau`, mark it for the *next* round, and log a "WALK" entry in the journal.

Phase E: Path Reconstruction (Lines 265–365)

- **Line 266:** If the `Destination` isn't in `bestArrival`, no path was found.
- **Line 271 & 289:** `reconstructPaths` loops backward through the rounds. It starts at the `Destination` and uses the `journal` to trace backwards.
- **Line 306-326 (The Fallback Scan):** If a station wasn't reached in the exact current round (e.g., we waited at a station), it scans earlier rounds to find when we actually arrived there.
- **Line 357:** `if !foundHere { currRound-- }` — The critical backtrack logic. If the previous leg wasn't in the same round (e.g. crossing a transit/walk boundary), we step back one round to find the next journal entry until we reach the `Origin`.

3. Dry Run (Small Example)

Scenario:

- Search: Station **A** to Station **C** departing at 09:00.
- **Train 1 (T1):** Departs **A** at 09:30, Arrives **B** at 10:30.
- **Train 2 (T2):** Departs **B** at 11:00, Arrives **C** at 12:00.

Initialization (Round 0)

- $\text{tau}[0][A] = 09:00, \text{bestArrival}[A] = 09:00.$
- $\text{marked} = \{A\}$

Round 1 (Direct trains)

1. **Route Collection:** marked contains A. Train 1 passes through A. $\text{queue} = \{\text{Route } 1\}.$
2. **Ride-Forward (Route 1):**
 - **At Station A:** We arrive at 09:00. We scan trips. T1 departs at 09:30. $\text{dep} \geq 09:00$, so we board T1. $\text{currentTrip} = \text{T1}.$
 - **At Station B:** We are on T1. T1 arrives at 10:30. This is better than ∞ .
 - $\text{tau}[1][B] = 10:30, \text{bestArrival}[B] = 10:30.$
 - $\text{journal}[1][B]$ records: Boarded at A, Route 1, Trip T1.
 - $\text{newMarked} = \{B\}.$
3. **Footpaths:** No footpaths.
4. marked becomes $\{B\}.$ End of Round 1.

Round 2 (One Transfer)

1. **Route Collection:** marked contains B. Train 2 passes through B. $\text{queue} = \{\text{Route } 2\}.$
2. **Ride-Forward (Route 2):**
 - **At Station B:** $\text{arrivalHere} = 10:30$ (from Round 1). We scan trips. T2 departs at 11:00. $\text{dep} \geq 10:30$, so we board T2. $\text{currentTrip} = \text{T2}.$
 - **At Station C:** We are on T2. T2 arrives at 12:00. This is better than ∞ .
 - $\text{tau}[2][C] = 12:00, \text{bestArrival}[C] = 12:00.$
 - $\text{journal}[2][C]$ records: Boarded at B, Route 2, Trip T2.
 - $\text{newMarked} = \{C\}.$
3. **Footpaths:** No footpaths.
4. marked becomes $\{C\}.$ End of Round 2.

(Assuming $\text{MaxRounds} = 2$ for brevity, loop ends).

Path Reconstruction

1. Start at Destination **C**.
2. Look in $\text{journal}[2][C]$. Found entry: boarded at **B** via T2.
 - Leg added: $B \rightarrow C$ (T2).
 - boardStop (B) was not reached in round 2. So currRound decrements to 1.
3. Current station is now **B**.
4. Look in $\text{journal}[1][B]$. Found entry: boarded at **A** via T1.
 - Leg added: $A \rightarrow B$ (T1).
 - currRound decrements to 0.
5. Current station is **A**. This is the Origin! Loop exits.
6. **Final Path returned:** $[A \rightarrow B, B \rightarrow C]$. Arrival Time: 12:00. Transfers: 1.